

UNIVERSITEIT VAN PRETORIA
UNIVERSITY OF PRETORIA
YUNIBESITHI YA PRETORIA

COS700 Research Report

A Per-User Behavioural Baseline Framework for Insider Threat Detection in Resource-Constrained Environments

Student number: u20748052

Supervisor(s):
Prof. Jan Eloff

Author: Selepe Sello

Live dashboard: `https://cos700research.tebogoyungmercykay.co.za`
Source code: `https://github.com/TebogoYungMercykay/insider_threat_cos700`

November 2026

Abstract

Insider threats are slow to detect and costly. In 2024, breaches by malicious insiders took an average of 287 days to identify and contain. Lightweight detectors that score every employee against one organisation-wide baseline fail in two directions at once. Users who are busy for good reason, such as IT administrators, fill the queue of alerts that a security analyst must check, while an insider whose behaviour changes slowly, within population norms, is never found. This report designs, builds and evaluates a lightweight per-user behavioural baseline framework. It profiles each user from their own history, falls back to a role-group profile when a user's history is too short (never needed on this dataset), and scores each day's departures from that profile with a single anomaly model.

The framework was evaluated on the full CERT Insider Threat Test Dataset r6.2 (4 000 users, 135 million log events) at an alert budget of 0.1% of user-days, a user-day being one user's activity on one calendar day. It found 21 of 38 low-and-slow malicious days and both insiders active in the evaluation period, against 8 days and one insider for the SMOTE-balanced Random Forest, the population-level detector the proposal set as the comparison because earlier work on this data found it the strongest classifier. It spent 3.5% of its alerts on IT administrators instead of 22.6%, and trained in about eight seconds instead of about five minutes. The same anomaly algorithm with a population reference found 3 days, which shows the gain comes from comparing each user with their own history. Across ten repeated runs the framework found 20 or 21 of the 38 days every time, and its advantages in recall (the share of malicious days found) and administrator alert share over every population detector held in all ten. A personal baseline is stable after about 30 days of history and useful after about 60, so the 14-day default for the personal history required before a user is judged against it is too low. The framework's explanations name a feature the insider actually used as the top reason on 95% of malicious days, against at most 50% for the Random Forest's SHAP (Shapley additive explanation) values over the same days. The evaluation period holds only two insiders and the data is synthetic, and both limits are discussed alongside the results.

Keywords:

insider threat detection, behavioural anomaly detection, per-user baseline, user and entity behaviour analytics, role-aware normalisation, alert fatigue, CERT r6.2, one-class SVM, explainability, resource-constrained deployment.

1 Introduction

Insider threats are among the slowest security incidents to detect and the most expensive to suffer. In IBM's 2024 study of 604 breached organisations, attacks by malicious insiders took an average of 287 days to identify and contain, against 258 days for breaches overall, and were the costliest initial attack vector at an average of USD 4.99 million [1]. The difficulty is built in. An insider already has a valid login and valid access, so the defences at the network edge do not apply, and detection comes down to telling malicious use of that access from routine use of it [2, 3].

Lightweight detectors, including the published detectors for the CERT data (a public synthetic dataset of employee logs, Section 4) compared in this report [4, 5], solve that problem by learning one picture of "normal" from the whole workforce and flagging whoever departs from it. This picture is called a baseline. A single shared baseline is cheap and simple, but it fails in two directions at once. Employees whose work is naturally intense, such as IT administrators who touch many machines, connect devices and work odd hours, are always unusual compared with the population, so they fill the alert queue, the list of flagged days an analyst works through, while doing their jobs. Security analysts, the people who check each alert, learn to discount false alarms they see every day [6]. On the other hand, an insider who changes their behaviour gradually, and keeps every individual day within what the population considers ordinary, is never found at all, even when the change is large compared with their own history. This is the "low-and-slow" pattern of data theft documented in the CERT insider threat casebook [2].

Commercial User and Entity Behaviour Analytics (UEBA) platforms address this by profiling each user and their peer group individually [7]. Academic work has likewise moved towards user-centred and per-user modelling of the CERT data [8, 9]. Both routes, however, need equipment and software that an organisation has to buy, run and staff, and this report calls an organisation that cannot do so resource-constrained. The routes need either a SIEM (a security information and event management system, which collects an organisation's logs centrally) feeding a licensed analytics platform, or a large neural network trained on a live stream of logs. There is no published, lightweight system that builds per-user baselines from exported log files on ordinary hardware and is tested against the population-level practice it would replace.

This report designs, builds and evaluates such a framework. Each user's normal behaviour is summarised from their own history and from their role group. The day's activity is then expressed as a list of deviations from that summary, one per measured quantity, called a deviation vector. A single

anomaly model, a model that learns what normal data looks like without any labels and scores how far a new day is from it, scores those vectors. Because the model sees "how unusual is this for this person" rather than raw activity, one cheaply trained model behaves as a per-user detector. The framework is evaluated on the full CERT Insider Threat Test Dataset r6.2 [10, 11] (4 000 users, 135 million log events) against the Random Forest, a vote of many decision trees, that earlier work found to be the strongest population-level classifier (a model that learns from labelled examples) for this data [4], and against control detectors that change only one thing at a time, so that any difference can be traced to its cause.

Contributions. This report makes the following contributions:

1. A lightweight per-user behavioural baseline framework, made of a Profile Builder, a Role-Aware Normaliser and an Anomaly Scorer, that trains on the full 4 000-user dataset in about eight seconds on a laptop.
2. A data pipeline that reads the 93 GB of raw r6.2 logs once and reduces them to an 8.6 MB table of one row per user per day in 3.3 minutes on 32 GB of memory, frozen with roles, labels and the date split into an 8.7 MB dataset, with exact day-level labels from the official answer key.
3. A controlled comparison at a fixed alert budget (a fixed share of days that may be flagged) on one date-ordered split, of 13 detectors on the full population and 45 detector settings (36 framework variants and 9 comparison detectors) on a 400-user sample, including the same algorithm with a population reference, a no-model detector that ranks by activity volume, and four of the classifier families Noever compared [4].
4. Three follow-up studies, on consistency across ten repeated runs with different random choices (seeds), on the history a personal baseline needs (SQ3), and on whether each system's explanations name what the insider did (SQ5).
5. A record of the negative and unexpected results, and a review of the research proposal against what was built.

Structure. Section 2 states the problem and questions, Section 3 the method, Sections 4 and 5 the background and related work, Section 6 the results with their limits and a review of the proposal, and Section 7 the conclusions.

2 Problem Statement

Organisations without commercial security platforms judge every user against one baseline for the whole workforce, and two failures follow. A high-activity user such as an IT administrator is flagged because their activity is unusual for the population though normal for their role. An insider who spreads data theft over many ordinary days is not flagged, since no single day exceeds a population threshold, though their behaviour has changed sharply [2].

Both failures have one cause. The detector has no picture of what is normal *for this individual in this role*. The gap is the absence of a published, lightweight framework that builds individual baselines from standard log exports, with the role group as fallback when a person's history is too short, without costly equipment. The scope is behavioural features from logs on one public dataset, CERT r6.2.

2.1 Research Questions

The hypothesis is that scoring each user-day against the user's own baseline, with role-group fallback, finds more low-and-slow insider activity and spends fewer alerts on high-privilege users than a population baseline, at a cost a laptop can carry. Five sub-questions test it, answered in Sections 6.2, 6.3, 6.6, 6.7 and 6.8. In SQ2, role-aware normalisation means the whole framework, and the role-only variant is reported separately. SQ5's investigation time was not measured directly, so Section 6.8 uses two stand-in measures.

SQ1 To what extent does a per-user behavioural baseline reduce missed detections of low-and-slow insider behaviour compared to a population-level baseline?

SQ2 To what extent does role-aware normalisation reduce the false positive rate for high-privilege legitimate users compared to population-level evaluation?

SQ3 What is the minimum amount of historical log data needed to construct a stable and reliable individual baseline?

SQ4 How does the computational cost of per-user baseline construction compare to population-level model training, and is it feasible in a resource-constrained deployment?

SQ5 What additional context does a per-user deviation score give a security analyst relative to a population-level score, and does it reduce investigation time?

3 Methodology

The research takes the stance that a claim is accepted only when a measurement supports it, and it uses a quantitative, controlled experiment to produce those measurements. Quantitative means every conclusion rests on measured numbers. Controlled means that when two systems are compared, only one thing differs between them. A per-user framework and a set of population-level detectors are trained on the same data, scored on the same later period that none of them saw in training, and compared on measures fixed in advance at the same alert budget. The software was built in repeated rounds, and each round was checked by automated tests that lock in specific ways a comparison on rare events can report a misleading number (Section 3.7). The detectors and measures named in this section are defined in Section 4. Figure 1 summarises the work as five steps. Step 1 reduces the raw logs to one row per user and day, and Step 2 attaches roles and labels and splits the rows by date (Section 3.1). Step 3 fits the per-user framework (Section 3.2) and Step 4 the population detectors it is compared with (Section 3.3). Step 5 scores every detector at the same alert budget (Section 3.4) and runs the follow-up studies (Section 3.5).

3.1 Data Preparation

Step 1 Reading the logs. The extracted r6.2 logs total 93.2 GB (binary units throughout this report, so 1 GB is 1024 times 1024 times 1024 bytes), of which the web log alone is 84 GB, and cannot be loaded into 32 GB of memory. Every feature the framework uses is a count, a sum or a count of distinct values per user per day. A feature is one number that describes one aspect of a user's day, such as the number of emails sent. Such features can be computed in a single pass through each file, holding in memory only one row per user-day rather than every event. The log reader therefore cuts each file into pieces that start and end on whole lines, reads the pieces in parallel, takes the calendar day and hour directly from the fixed-width time stamps, and adds the piece totals together. The one feature that cannot be added across pieces, the number of distinct web hosts, is carried as a compact set of host identifiers per user-day and combined when the pieces are merged. On the 14-core laptop, with 12 parallel workers (a worker is one copy of the program running at the same time as the others, and the count changes only the speed), this reduces the 135 million events to 1 394 010 user-days in 3.3 minutes. With 4 workers the same pass took 15 minutes and used at most 2.8 GB of memory. The table behind every result in this report was built with 8 workers, and its content does not depend on the number of workers.

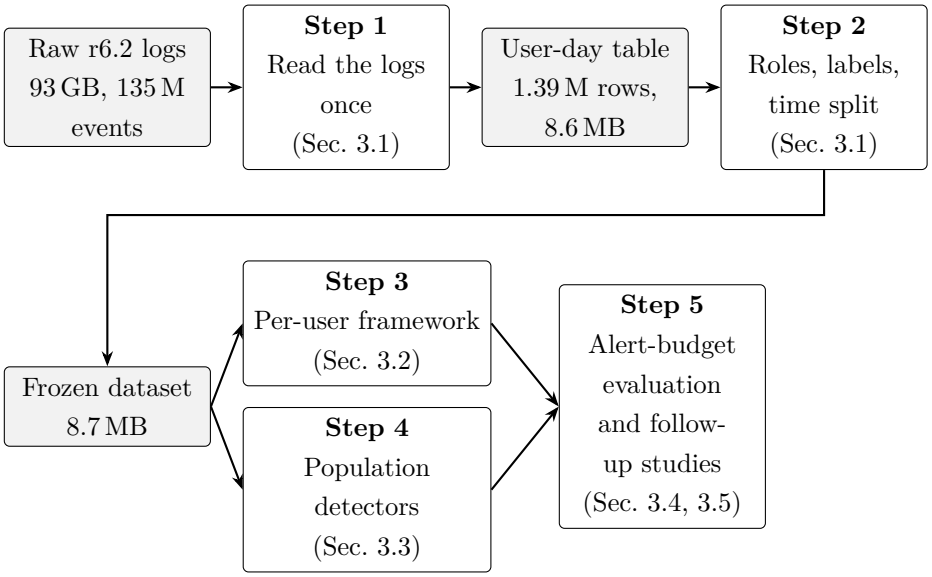


Figure 1: The pipeline in five steps, each described in the section named in its box. The raw logs are read once (Step 1), reduced to one row per user and day, and frozen, meaning written once, fingerprinted and never changed, with roles, labels and the train/test split into a single 8.7 MB file (Step 2). The per-user framework (Step 3) and the population detectors it is compared with (Step 4) are trained from that file alone, and every detector is scored under the same evaluation (Step 5).

Tests require the result from pieces to equal a single-pass result exactly, and two builds of the table on different days carry the same fingerprint of their content (a hash).

The features. Each user-day carries thirteen behavioural features (Table 1). Four of them were constant or broken against the real data in an early version because of coding mistakes, which were corrected and locked in by tests before any experiment was run (Table 14). An email is external if any recipient in the to, cc or bcc fields lies outside the organisation's own domain, which is read from the LDAP directory.

Step 2 Labels. A label marks a user-day as malicious or benign (not malicious), and the labels come from the official answer key. Each malicious log line is mapped to its user and calendar day, which gives exact day-level labels, 44 malicious user-days belonging to five insiders. Three earlier labelling

Table 1: The thirteen per-user-day behavioural features.

Log	Feature	Definition
logon	n_logon, n_logoff	Logon and logoff events
	n_afterhours_logon	Logons before 07:00 or from 19:00
device	n_device_connect	Removable-device connections
file	n_file_events	File operations
	n_file_to_removable	File operations whose target is removable media
http	n_http_events	Web requests
	n_http_upload	Web uploads
	n_distinct_domains	Distinct web hosts visited
email	n_email_sent	Emails sent ("Send" rows only)
	n_email_external	Sent emails with at least one external recipient
	n_email_attachments	Attachments on sent emails
	total_email_size	Total bytes of sent email

mistakes (reading every release's key, counting the 24 recipients of the scenario 3 email as attackers, and marking every day between an insider's first and last event) are recorded in Table 14.

Step 2 Roles and the date split. Each user-day is joined to the role, department and supervisor recorded for that user in the LDAP directory. The data is then split by date at 70% of the 516 calendar days (the first 361 days train, the rest test). The training part, 990 765 user-days up to 28 December 2010, is what every detector learns from. The test part, 403 245 user-days from 29 December 2010, is what every detector is scored on and never sees in training. A random split instead of a date split would let a detector learn from days that follow the ones it is scored on. The split has a result that shapes everything that follows. Sorting insiders by the span of their malicious activity, longer than three weeks is low-and-slow and shorter is a burst, gives three burst insiders and two low-and-slow ones. All three burst insiders act before the split and both low-and-slow insiders act after it (Table 4). The test window therefore contains 38 malicious days, all low-and-slow, and the training window contains six, all burst. The supervised detectors, which learn from labelled examples, must learn what malicious

looks like from six examples of a different kind of attack. Moving the split to put a burst insider in the test window would leave them with no malicious training rows at all, so the limit is reported rather than engineered away. It also means the test window measures exactly the behaviour SQ1 asks about. An active day is a day on which a user has at least one logged event, and every day count in this report is in active days. Every one of the 4 000 users has at least 21 active days in the training window (247.7 on average), and 3 744 of them appear in the test window. The other 256 have no logged activity after the split date.

3.2 The Per-User Framework

The framework has the three parts the proposal specifies. Figure 2 gives the whole procedure.

Profile Builder. For every user and every feature, the mean, the standard deviation (the usual spread of the values) and the 25th, 50th, 75th, 90th and 99th percentiles (the 90th percentile is the value that 90 percent of the user's days fall below) are computed from that user's training days. The percentiles are stored so that an explanation can say what a typical and a busy day look like for this person.

Role-Aware Normaliser. The same statistics are computed for every role group and for the whole population. A user with fewer than 14 active days of history (the cold-start threshold) is measured against their role group, and a role absent from the training window falls back to the population. Every r6.2 user has a role in the directory, so no user lacks one. Because every user has at least 21 training days, the fallback is never triggered in the main experiments. It is used, on purpose, by the history study of Section 6.6. Whose normal a day is compared with is called the reference frame, and it is a single setting named *scope* (`population`, `role` or `personal`). A second setting, the *statistic*, chooses the z-score or the median form of the deviation (both defined in Section 4). These two words name every framework detector in the tables. The scope setting means so the three frames share one piece of code and differ in nothing else. This is what makes the comparison in Section 6.4 a clean test of the reference frame.

Anomaly Scorer. Each user-day becomes a deviation vector, one deviation z_f (z-score) or z_f^{median} (median form) per feature as defined in Section 4, measured against the row's reference profile. A spread of zero, for a feature

the user has never shown, is raised to one, so the first-ever write to removable media counts as a deviation of one per file rather than a division by zero. One anomaly model is trained ("fitted") on the training deviation vectors and scores the test vectors. Because those vectors already express "how unusual for this person", a single shared model behaves as a per-user detector, and no per-user model has to be trained or stored. The training time of LOF and the One-Class SVM (both defined in Section 4) grows with the square of the number of rows, so they could not be fitted on 990 765 rows on the laptop. Every anomaly model, Isolation Forest included, is therefore fitted on the same random sample of 50 000 training rows, drawn with a fixed seed. The population anomaly controls of Section 3.3 use the same sample size. Profiles are always built on the full training window, and every test row is scored.

The ranking always uses the model's own score. The 0 to 1 score shown in the dashboard is for display only and keeps the same order (an early version that capped scores at 1.0 tied the whole top of the ranking, Table 14).

3.3 Comparison Detectors

Every detector works the same way from the outside. It is fitted on the training part and returns one score per test row, higher meaning more suspicious. No detector sets its own cut-off. Table 2 lists the families.

The supervised baselines are a Random Forest of 300 trees, a decision tree (a single tree of threshold rules), a histogram-based gradient boosting classifier (an ensemble of shallow trees, each fitted to the errors of the previous ones, since the exact learner could not fit the SMOTE-rebalanced training set, which is doubled to two million rows, in reasonable time) and a logistic regression (a model that adds up weighted features to give a probability) on features rescaled to a common range. All four are SMOTE-balanced. Detector names in the tables and figures follow one pattern. `rf`, `dt`, `gb` and `lr` are the Random Forest, the decision tree, gradient boosting and logistic regression. `pop-iforest`, `pop-lof` and `pop-ocsvm` are the Isolation Forest, LOF and One-Class SVM with a population reference. A framework detector is named `peruser-<algorithm>-<scope>-<statistic>`, so `peruser-ocsvm-personal-zscore` is the One-Class SVM on personal z-scores and `peruser-iforest-role-zscore` is the Isolation Forest on role-group z-scores, and a trailing `-mag` marks the size channel. `volume` ranks days by total activity and `random` ranks them at random. In short labels, OCSVM stands for One-Class SVM and iForest for Isolation Forest. The population unsupervised detectors are what make the main comparison mean something.

Input: training user-days D_{train}, test user-days D_{test}, scope, statistic, anomaly model M, cold-start threshold $h_{\min}$

Output: one anomaly score per test user-day, and its explanation

```
# Profile Builder + Role-Aware Normaliser
# (mean, std and p25..p99 per group)
P_user = stats_by(D_train, "user")
P_role = stats_by(D_train, "role")
P_pop  = stats(D_train)

def reference(row):   # scope = personal
    # (role scope: P_role, population: P_pop)
    # history = active training days
    if history(row.user) >= h_min:
        return P_user[row.user]
    if row.role in P_role:
        return P_role[row.role]
    return P_pop

def deviations(rows):
    c, s = centre_scale(reference, rows)
    s[s <= 0] = 1.0   # never-seen feature
    return (rows.X - c) / s

# Anomaly Scorer: one model on deviations
M.fit(subsample(deviations(D_train), 50000))
Z = deviations(D_test)
# library scores higher = more normal, so flip
score   = -M.decision_function(Z)
explain = top_k(abs(Z), k=3)
```

Figure 2: The per-user framework as one procedure, fit and score. Profiles are built per user, role and population, each row becomes a deviation vector against its reference profile, and one anomaly model scores the vectors.

Table 2: The detector families in the comparison.

Family	**Detectors**	**Purpose**
Floors	`random`, `volume`	No-model detectors. `volume` scores a day by the sum of its thirteen feature values and `random` gives every day a random score. A detector that cannot beat them is not worth its extra effort
Population, supervised	`rf`, `dt`, `gb`, `lr`	Random Forest, decision tree, gradient boosting and logistic regression, four of the classifier families Noever compares [4]. `rf` is the proposal's baseline. Trained on a SMOTE-rebalanced set (Section 4), raw features
Population, unsupervised (no labels)	`pop-iforest`, `pop-lof`, `pop-ocsvm`	Isolation Forest, LOF and One-Class SVM with a population reference, the control that isolates the reference frame
Per-user framework	`peruser-<alg>-<scope>-<stat>`	The framework across 3 algorithms, 3 scopes, 2 statistics, with and without a size channel (defined below)

`pop-ocsvm` and the framework's `peruser-ocsvm-personal-zscore` run the same One-Class SVM with the same parameters on the same rows. They differ only in whether each row is expressed relative to the population or relative to the user. Without that control, a framework that beat the Random Forest would have changed two things at once, the algorithm and the reference frame. The *size channel* is an optional variant that adds the raw features, rescaled against the population, to the deviation vector, so that the anomaly model can require a day to be both unusual for the person and large in absolute terms.

3.4 Evaluation Method

Alert budget. An alert budget is the share of test user-days a detector may flag. Every detector is evaluated at a budget of 0.1%, which is 403 alerts over five months across 4 000 users, about three a day. Budgets from 0.01% to 5% are also tried to show the trade-off.

Ties. Ties, where many rows share one score, are the normal case on this data. A Random Forest trained on six malicious rows gives most rows a probability of exactly zero. The first version flagged every row tied at the cut, which on the 400-user sample of Section 3.6 let the Random Forest flag all 40 360 of its test rows against a 1% budget of 404. Ties are now broken by a random shuffle fixed by the seed, so exactly the budgeted number of rows is flagged, and the choice among equals does not depend on the detector or on the row order. The share of a detector's budget decided this way is reported as its *tie fraction*.

Metrics. The proposal's measures are reported at the budget, namely recall, precision, F1, the false positive rate (the share of benign days wrongly flagged) overall and among high-privilege users, and ROC-AUC, all defined in Section 4. Average precision ranks detectors by the very top of their queue and is the main ranking measure, while recall and insider-level recall at the budget say how much of the malicious activity the budget catches, and the main claims rest on those. SQ2 is answered mainly by the alert share below, because that is what fills the analyst's queue, with the false positive rate given as well. Four more measures were forced by the data:

- **Insider-level recall**, whether each insider was found at all. Day-level recall over 38 malicious days moves in steps of 2.6 points, since one day is 1 in 38. Whether the person was found is what an analyst acts on.
- **High-privilege alert share**, the share of the alert queue spent on high-privilege users. These users are 2.4% of the test rows, so a small subgroup false positive rate can still mean they fill most of the queue. The alert share is what alert fatigue is made of.
- **Alerts ahead**, also called queue depth, the number of user-days ranked above an insider's best-ranked malicious day, which needs no cut-off. This is how far down the queue an analyst must read to reach that insider, and it stands in for investigation effort in SQ5.
- **Detection delay**, the days between an insider's first malicious day and the first malicious day flagged at the budget. The problem statement is about how long insiders go undetected, and this measures it directly.

High-privilege users are identified from the LDAP role by matching administrator role names. In r6.2 this subgroup is exactly the ITAdmin role, 100 users and 2.43% of test user-days. The proposal named database administrators, system operators and security analysts, and none of these roles exists in r6.2. Every measure is computed on the full test part, never a sample. A 5 000-row sample of 403 245 rows would contain 0.47 malicious days on average.

3.5 Follow-up Studies

Seed consistency. A seed is the number that fixes every random choice a run makes, so the same seed gives the same run again. The main comparison uses one seed. The seed study re-runs seven detectors under ten seeds. These are the framework, its role-scope and Isolation Forest variants, the two population anomaly controls, the Random Forest and the volume floor. Each seed re-draws the training sample, the forests, SMOTE and the tie-break, while the split and features stay fixed. The framework is compared with each other detector on average precision, recall and high-privilege alert share with the paired tests of Section 4, corrected with the Holm procedure across all 18 comparisons (6 other detectors times 3 measures) at the 5 percent level.

History length. To find how much history a personal baseline needs, each user's training data is cut to their most recent N active days, for $N \in \{3, 7, 14, 30, 60, 90, 120, 180\}$ and the full window (248 active days on average). This imitates an organisation that has been collecting logs for N days when monitoring starts. The framework is refitted on the shortened data and scored on the unchanged test window. The cold-start threshold is lowered to N where necessary, so every user is really measured against an N-day personal profile rather than silently falling back to their role. Two properties are measured. *Stability* has two parts, how far the N-day per-user means sit from the full-history means, in full-history standard deviations, and the share of the test window's *benign* user-days that show any feature beyond $|z| = 3$ (three spreads from the usual value, Section 4) under the N-day profile. The latter is the load of false deviations an analyst would be left with. *Usefulness* is detection at the 0.1% budget, compared with the role baseline, which needs no personal history at all. Both the One-Class SVM and Isolation Forest scorers are run under three seeds.

Explanation accuracy. Every answer-key row is an exact copy of a malicious log line. Replaying those rows through the same code that reads the logs gives, for each malicious user-day, exactly which features the malicious events changed. This is the ground truth, the official record, for "what the insider did that day", with no hand-made mapping. Each of the 38 malicious test days is explained five ways, each the top three features an analyst would be shown, and every explanation is scored against this ground truth:

- the framework's (largest $|z|$ against the user's own history).
- the same against the role group.
- the same against the population, the control that isolates the reference frame from the explanation method.
- per-row SHAP values (Section 4) of the proposal's Random Forest, ranked by size ignoring sign.
- the same SHAP values restricted to features that push towards an alert, since a negative value is a reason against one.

The measures are hit@1 (the top-named feature is one the insider used), hit@3 (any of the top three is) and precision@3 (the share of the top three that are). They are compared with the exact expected rate of a random explanation that names k of the 13 features, $1 - \binom{13-g}{k} / \binom{13}{k}$ for g relevant features, with $k = 1$ for hit@1 and $k = 3$ for hit@3, averaged over the days.

3.6 The Code and Its Settings

The system is written in Python 3.14 with pandas 3.0, NumPy 2.5, scikit-learn 1.9 [12], imbalanced-learn 0.14 (SMOTE), SHAP 0.52 and SciPy 1.18, and has a dashboard built with Streamlit, a Python framework for browser applications. The source code is public at the address on the title page, and its documentation lists the command behind every table in this report. Table 3 lists the defaults. All experiments ran on a consumer laptop (14 cores, 32 GB of memory, no graphics processor) under Linux. Every run writes a record file listing the dataset, split, features, seed, machine and elapsed time, so each table in this report traces back to the run that produced it. The frozen dataset's record also holds the fingerprint of the user-day table it was cut from. A rebuild of the user-day table on 29 October 2026 produced the same hash as the table recorded on 21 October, and a rerun of the main and baseline comparisons on the rebuilt data gave every detection measure again to four decimals. Only elapsed times differed (Section 6.7). The main comparison, the baselines and the three studies were run on 21

and 22 October 2026 and again on 29 October, and the comparison of the framework's parts and the rolling-feature comparison on 8 September 2026, the day after the data pipeline was first tested.

Table 3: Default configuration.

Parameter	**Description**	**Default**
Scope	Reference frame for deviations	`personal`
Statistic	Deviation form	z-score
Anomaly Scorer	Anomaly model on deviations	One-Class SVM (RBF kernel, $\gamma = 1$ over the number of features times their variance, the library's scale rule)
Contamination / ν	Expected share of unusual rows	0.02
Cold-start threshold	Days before a personal profile is trusted	14
Training sample	Training rows for every anomaly model	50 000 (seeded)
Isolation Forest	Trees	200
LOF	Neighbours	25
Random Forest	Trees, depth	300, unlimited
SMOTE	Neighbours	$\min(5, n_{\text{pos}}-1)$, 5 here since $n_{\text{pos}} = 6$ malicious training rows
Alert budget	Share of test rows flagged	0.1% (403 alerts)
Low-and-slow threshold	Span of malicious activity	> 21 days
Seed	All random choices	42 (ten in the seed study)

Choosing the settings. The anomaly algorithm and the feature set were chosen by experiment, as the proposal requires, from the 45-detector comparison in Section 6.4. That comparison was run on a fixed sample of 400 users, drawn so that every role keeps its share and all five insiders are present. The other settings are the defaults of the software library or follow from the data. The contamination rate, the share of training rows the anomaly model is told to treat as unusual, is 0.02, inside the range of alert budgets swept (0.01 to 5 percent). The 14-day cold-start threshold is 14 active days, about three weeks of work, and SQ3 tests it directly. The main results used 14, which never fell back, and 60 is the recommendation that SQ3 produces.

3.7 Testing the Code

The code carries 72 automated tests. Sixty-two run in seconds without CERT data, and ten slow tests recompute features from the 93 GB of raw logs and check them against the user-day table. Twenty-seven of the fast tests cover the model and the evaluation. They check that the profile stores the percentiles the proposal specifies, that each scope uses its own reference, that a never-seen event is a deviation and not a division by zero, that ranking uses the model's own score, that exactly k rows are flagged even when scores are tied, that rolling features use only past days, that profiles are built from training rows only, and that the framework at population scope reduces to the population detector. Twenty-five cover the data pipeline, including that reading in pieces gives the same result as one pass, the removable-media flag, external-recipient detection and labels restricted to r6.2 and to the named insiders. Ten cover the follow-up studies, including a hand-computed Holm adjustment, history shortening, and an answer-key replay that recovers exactly the features the events moved. One more defect was found while writing the follow-up studies. SMOTE was always seeded with 42 regardless of the run's seed, which was fixed before the seed study ran, and the seed-42 results were unchanged.

4 Background

4.1 Insider Threats and the Dataset

An insider threat is a current or former employee, contractor or partner who misuses allowed access to harm the organisation [2]. Surveys of the field group detection methods by the data they use and separate misuse that is sudden from misuse that unfolds slowly [3, 13]. Real labelled insider data is rarely released [3], so research relies on synthetic data, data generated by a program to look like a real organisation's logs. Most recent studies use the CERT Insider Threat Test Dataset [10, 13], generated by the CERT Division of the Software Engineering Institute at Carnegie Mellon University, with ExactData (a data-generation company), from models of how employees of a made-up organisation behave, with malicious scenarios written in by the dataset's authors [11].

Release r6.2, used here, covers 4 000 employees. The five event logs (logon and logoff, removable-device connect and disconnect, file operations, web browsing and email) run continuously from 2 January 2010 to 1 June 2011, 516 calendar days, and every line is one event stamped to the second with its user and computer (Table 5). Eighteen monthly snapshots of the organisation's LDAP directory (Lightweight Directory Access Protocol, the employee directory), from December 2009 to May 2011, record each employee's role, department and supervisor. There are 46 distinct roles. An answer key lists every malicious log line, and Table 4 summarises the five scenarios it records, one insider each.

Only 44 of 1 394 010 user-days are malicious, about one in 31 700, and the insiders are not spread evenly across the calendar (Section 3.1).

4.2 Behavioural Baselines

A behavioural baseline is a description of normal activity against which a new observation is compared. For a feature x_f observed on one user-day, the standard score, or z-score, against a reference with mean μ_f and standard deviation σ_f is

$$z_f = \frac{x_f - \mu_f}{\sigma_f}.$$

It says how many spreads above or below the usual value today lies. The choice of reference, called the reference frame, is the subject of this research. A *population* baseline estimates μ_f and σ_f over every employee. A *role* baseline estimates them over the employee's role group. A *personal* baseline

Table 4: The five r6.2 insiders, as recorded in the official answer key. "Events" is the number of malicious log lines belonging to the insider. "Days" counts distinct malicious user-days. "Span" is the number of whole days elapsed between the first malicious event and the last.

Scenario	Insider	Role	Malicious activity	Events	Days	Span	Split
1	ACM2278	Salesman	After-hours logons, removable drive, uploads to wikileaks.org	22	3	5 d	train
2	CMP2946	Salesman	Job-site browsing, job-search email to competitors, thumb-drive use	242	29	56 d	test
3	PLJ1771	ITAdmin	Keylogger web visits and download, removable drive, after-hours logons, an email that reached 24 colleagues	18	2	1 d	train
4	CDE1846	Elec. Engineer	Logons to other users' PCs, file opens, documents emailed to a personal address	134	9	63 d	test
5	MBG3183	Elec. Engineer	Four uploads to a file-sharing site in one minute	4	1	0 d	train

estimates them over the employee's own history.

Behavioural counts are mostly zero with a few large values. Most users never write to removable media, and when they do it is in bursts. A single large day inflates σ_f and hides the next one. The median alternative, which is less affected by extreme values, replaces the mean with the median (the middle value) and the standard deviation with the interquartile range (IQR, the spread of the middle half of the values), scaled so that it matches σ for

Table 5: The r6.2 logs as extracted. Sizes are in binary units. Event counts are line counts of the extracted files, less the header line.

Log	Events	Size	What is drawn from it
logon	3 530 285	230 MB	Logons, logoffs, after-hours logons
device	1 551 828	133 MB	Removable-device connections
file	2 014 883	1.24 GB	File operations, copies to removable media
http	117 025 216	84.0 GB	Web requests, uploads, distinct hosts
email	10 994 957	7.53 GB	Emails sent, external recipients, attachments, size
LDAP	18 snapshots		Role, department, supervisor, company domain
Total	135 117 169	93.2 GB	

bell-shaped data [14]:

$$z_f^{\text{median}} = \frac{x_f - \text{median}_f}{\text{IQR}_f / 1.349}.$$

4.3 Anomaly Detection Without Labels

An anomaly is an observation far from what is usual. Labelled insider activity is scarce, so anomaly detectors, which learn only from (mostly) normal data and need no labels, suit data with few labels [15]. Three are compared, as the proposal specifies. **Isolation Forest** [16] separates points by repeated random splits on one feature at a time. An unusual point is cut off from the rest in fewer splits, so the average number of splits needed to isolate a point is its score, flipped so that higher means more unusual. Its training time grows about in step with the number of rows, and it copes with features that repeat the same information, because each split looks at one feature. **Local Outlier Factor (LOF)** [17] compares how crowded the space around a point is with how crowded it is around the point's nearest neighbours (25 here), so points in much emptier regions than their neighbours' score high. **One-Class Support Vector Machine (SVM)** [18] measures the

similarity of two points with a function called a kernel, $k(\cdot, \cdot)$, and finds the flat boundary that separates the data from the origin (the point where every feature is zero) with the widest gap. A setting $\nu \in (0, 1]$ limits the share of training points allowed outside the boundary, and the signed distance from the boundary, $\sum_i \alpha_i k(x_i, x) - \rho$, is the score, where the x_i are the training points the model keeps (its support vectors), the α_i are weights set in training and ρ is a constant set in training. With a radial basis function (RBF) kernel, $k(x, x') = \exp(-\gamma \|x - x'\|^2)$, where γ sets how fast similarity falls with distance, the model is sensitive to the shape of the whole feature space, which matters when features are added (Section 6.4), and far from every support vector the kernel sum is zero and the score equals the constant ρ (Section 6.10). Its training time grows with the square of the number of rows.

4.4 Supervised Detectors and Rare Classes

A supervised classifier learns to tell malicious from benign directly from labelled examples. On the CERT data, Noever compared 88 algorithms from 16 families and found Random Forests, groups of many decision trees that vote, the strongest [4], which is why the proposal names a Random Forest as the population baseline. With malicious activity at 0.003% of user-days, the two classes are extremely unequal in size, and the proposal specified rebalancing the training set. The Synthetic Minority Over-sampling Technique (SMOTE) makes new malicious examples by placing points between a real malicious example and one of its k nearest malicious neighbours [19]. It is applied to the training part only, after the split, so no made-up row can carry information from the test window.

4.5 Measuring Detection of Rare Events

ROC-AUC, the area under the receiver operating characteristic curve, measures the probability that a random positive is ranked above a random negative. When benign days outnumber malicious ones about 10 600 to one, as in the test window, a detector can order the benign mass well, and so score a high ROC-AUC, while placing no malicious day anywhere near the top of the queue [20, 21]. Average precision (AP) is used instead. It is the precision averaged over every cut-off at which one more malicious day is found. Its chance level equals the share of malicious days, here about 10^{-4}, and it is decided mostly by the top of the ranking, which is the only part an analyst ever reads. A Random Forest gives a probability, an Isolation Forest a split

count and a One-Class SVM a distance, so cutting all three at one value compares a probability with two numbers that have no shared meaning. The comparison is therefore made at an *alert budget*, where each detector may flag the top b fraction of test user-days. This also reflects that an analyst team can investigate only a fixed number of alerts a day. At a budget, recall (the share of malicious days flagged), precision (the share of alerts that are malicious) and F1 (their harmonic mean) are well defined for every detector, and *lift*, the precision divided by the positive rate, says how many times better than random the alerts are.

4.6 Explanations and Statistical Tests

A flagged user-day with no reason attached must be investigated from scratch. SHAP splits a model's output among its input features using Shapley values, a rule for sharing a result fairly among the things that produced it, so that the feature contributions add up to the difference between this prediction and the average prediction [22]. TreeExplainer computes these values exactly and quickly for groups of trees [23]. The per-user framework has a more direct explanation available, the deviation vector itself. Its largest entries name the features on which today departs most from this user's normal.

Several parts of every detector involve chance, including sampling, building forests, SMOTE and tie-breaking, so a difference seen under one seed may be the result of that one draw. Re-running each detector under n seeds gives n paired results. The Wilcoxon signed-rank test [24] ranks the paired differences by size and tests whether the positive and negative ones balance, without assuming a bell-shaped distribution. It is the test recommended for comparing classifiers over paired runs [25]. The result is a p-value, the chance of seeing a difference at least this large if there were no real difference. With $n = 10$ seeds and every difference in the same direction, the exact two-sided p-value is $2/2^{10} \approx 0.002$, the smallest the test can give at this n. When m tests are run together, the chance that at least one is a false finding grows. The Holm-Bonferroni procedure [26] keeps that chance at the chosen level α (5 percent here) by comparing the i-th smallest p-value with $\alpha/(m - i + 1)$ and stopping at the first failure.

5 Related Work

Machine learning on the CERT data. Le et al. showed that the unit of counting (user-day, user-week or session) changes what detectors find [9]. Yuan et al. replaced hand-made features with a long short-term memory network (LSTM), a neural network for sequences, followed by a convolutional neural network (CNN) classifier [27]. These, like the classifiers Noever compared (Section 4), learn one decision rule from every user and apply it to every user. This work keeps the user-day unit and Noever's Random Forest as its comparison point, and changes only the reference frame.

Per-user and user-centred modelling. Tuor et al. ran deep neural networks that learn without labels on a stream of CERT r6.2 events, split each score into per-feature contributions, and reported gains over Isolation Forest, principal component analysis (PCA) and SVM baselines [8]. Commercial UEBA platforms such as Microsoft Sentinel take the same idea into production, profiling each user against their own history and their peers' [7]. Both need equipment, graphics processors for training on streaming data or a licensed SIEM and analytics platform, that a resource-constrained organisation lacks. The framework here pursues the same idea with simple summary statistics and one standard anomaly model, on exported log files. In interviews with security operations centre analysts, Alahmadi et al. found false positives to be the dominant complaint and that analysts come to distrust tools that raise them [6], which is why this report measures *who* an alert budget is spent on.

Explanations and forensic readiness. None of the works above scores its explanations against the answer key, and this report does. Shoderu et al. argue that insider-threat defence in software-as-a-service (SaaS) environments remains reactive, and propose building in digital forensic readiness, the capture of evidence early and in a form that holds up legally, following the ISO/IEC 27043 standard [28]. A per-user baseline that explains each alert against the user's own history is one source of such evidence.

Gap. The per-user approach exists only as large neural networks or commercial platforms. No published study builds a lightweight per-user baseline on exported log files and consumer hardware, compares it under a controlled design with the population practice it would replace, and scores its detection and explanations against the answer key.

6 Discussion

Unless stated otherwise, results are for the full 4 000-user population and the main seed (42), at the 0.1% alert budget, that is 403 alerts over 403 245 test user-days, which contain 38 malicious days belonging to two insiders. Section 6.5 shows that the conclusions hold across ten seeds.

6.1 Main Comparison

Table 6 compares the framework with every population detector and both floors.

The framework flags 21 of the 38 malicious test days (recall 0.553), against 8 for the Random Forest (0.211). It is the only detector to find *both* test-window insiders. Its alerts are 553 times more likely to be malicious than a random alert. It spends 3.5% of its alert queue on IT administrators against the Random Forest's 22.6%, and it trains in about eight seconds against about five minutes (Section 6.7).

The same-algorithm control isolates the cause. `pop-ocsvm` runs the same One-Class SVM on the same rows, with each row expressed relative to the population instead of relative to the user. It finds 3 malicious days and spends 20.8% of its alerts on administrators. Changing nothing but the reference frame therefore raises recall from 0.079 to 0.553 and cuts the high-privilege alert share to about one sixth.

Figure 3 shows that the ordering does not depend on the chosen budget. From 0.05% upwards the framework's recall exceeds every population detector's, and at 0.5% (about 13 alerts a day) it reaches 0.84. The Random Forest stays at 0.21 from 0.05% to 1% because its ranking runs out. Beyond its few confident rows, every remaining test row has probability zero, and 85% of its 0.5% budget is a random draw among ties. At the smallest budget, 40 alerts, the framework finds nothing. Section 6.10 traces this to a ceiling in the One-Class SVM's scores.

Table 6: Full population, 0.1% alert budget (403 alerts), seed 42. "Days" is the number of the 38 malicious test days flagged. "HP share" is the share of alerts spent on high-privilege (ITAdmin) users, who make up 2.43% of test user-days. Best value per column in bold. The role variant has the highest AP because AP is decided by the very top of the queue (Section 6.10). A ROC-AUC below 0.5 means worse than random ordering. Lift (precision over the base rate) is 553 for the framework and 211 for the Random Forest. The median-statistic and size-channel variants are not listed here, and their numbers appear in Section 6.3. Training cost is reported in Section 6.7.

Detector	**AP**	**ROC-AUC**	**Days**	**Insiders**	**HP share**
Per-user framework					
`peruser-ocsvm-personal-zscore`	0.0383	**0.967**	**21**	**2/2**	3.5%
`peruser-ocsvm-role-zscore`	**0.0433**	0.675	3	1/2	0.7%
`peruser-iforest-personal-zscore`	0.0030	0.921	3	1/2	7.7%
Population, unsupervised (same algorithms, population reference)					
`pop-ocsvm`	0.0364	0.446	3	1/2	20.8%
`pop-iforest`	0.0009	0.935	0	0/2	49.4%
`pop-lof`	0.0003	0.831	0	0/2	**0.0%**
Population, supervised (SMOTE-balanced)					
`rf` (proposal baseline)	0.0077	0.605	8	1/2	22.6%
`lr`	0.0284	0.864	2	1/2	9.7%
`gb`	0.0003	0.379	1	1/2	44.4%
`dt`	0.0001	0.500	0	0/2	2.7%
Floors					
`volume`	0.0284	0.821	2	1/2	4.0%
`random`	0.0001	0.445	0	0/2	2.0%

6.2 SQ1 Finding Slow Insiders

Every malicious day in the test window belongs to a low-and-slow insider, so the main recall answers SQ1 directly. The per-user baseline finds 2.6 times as many low-and-slow malicious days as the population Random Forest (21 versus 8) and 7 times as many as the same algorithm with a population reference (21 versus 3).

Figure 4 shows why. CMP2946's scenario (job-hunting followed by copying data to a thumb drive) never produces a day that is extreme for the organisation. This user made exactly 162 web requests on every one of their

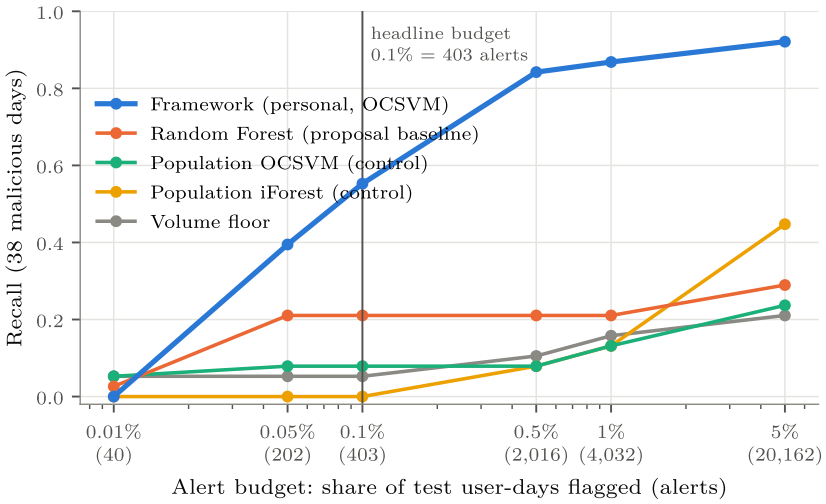


Figure 3: Recall of the 38 malicious test days at alert budgets from 40 to 20 162 alerts.

249 training days and about 164 a day on average during the test window, which is ordinary across 4 000 users. Against their own history, however, the scenario's days are sharp departures. The framework places 18 of their 29 malicious days inside the alert line, and the population-referenced model places none. CDE1846's malicious days, by contrast, are large bursts of emailed attachments that are extreme for anyone. Both reference frames catch the same 3 of their 9 days (22 March, 11 April and 25 April 2011), and the per-user baseline neither gains nor loses on them. The framework's advantage is therefore found exactly where the research question locates it, in behaviour that is unusual for the person but not for the population.

Alerts ahead and delay. Table 7 reports, for each insider and across all ten seeds of Section 6.5, how many user-days rank above the insider's first appearance in each detector's queue, and how long after their first malicious day they were first flagged. Every population detector buries one of the two insiders thousands of alerts deep. The Random Forest ranks CMP2946's scenario highly, but places CDE1846 about 24 000 alerts down, roughly sixty times its budget, and never flags them. The same-algorithm control and the volume floor do the reverse. The framework is the only detector that brings both insiders inside its first 200 alerts in every seed. It flagged CMP2946 on the first day of their scenario. It flagged CDE1846 29 days in, on the first of their malicious days to involve a burst of emailed attachments, the same day on which the role-scoped variant, the Isolation Forest variant and

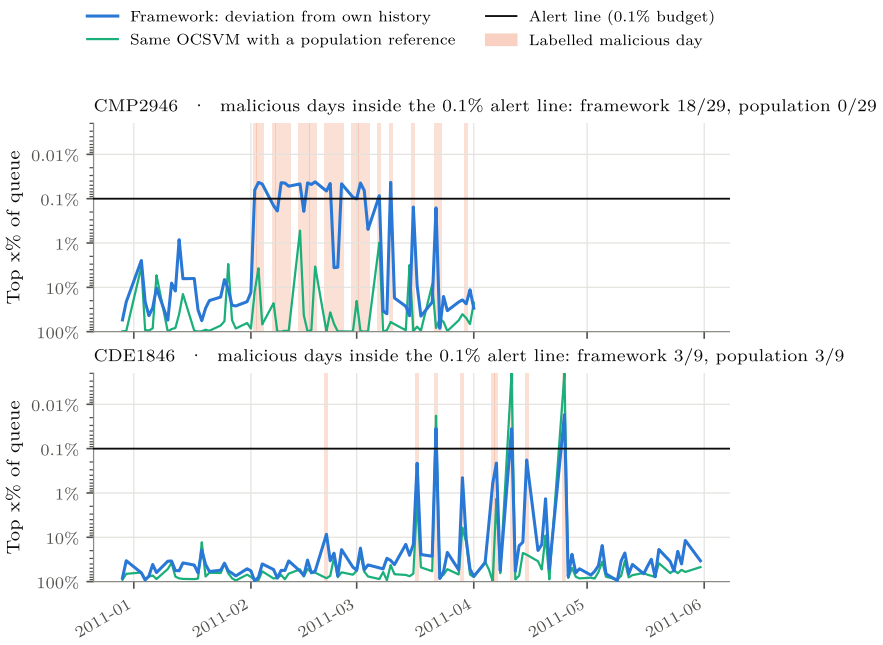


Figure 4: Daily queue position of the two low-and-slow insiders over the test window, under the framework and under the same One-Class SVM with a population reference. Shaded days are labelled malicious. The black line is the alert line, the queue position at which the 0.1% alert budget is cut.

the population One-Class SVM first reached them (median over seeds). The volume floor reached them 20 days later.

6.3 SQ2 False Alerts on Administrators

High-privilege users are 2.43% of the test user-days. Figure 5 shows how much of each detector's queue they occupy. Four population detectors overspend on them by nine to twenty times. 49.4% of `pop-iforest`'s alerts, 44.4% of `gb`'s, 22.6% of the Random Forest's and 20.8% of `pop-ocsvm`'s go to administrators doing their jobs. Every per-user variant without the size channel sits between 0.5% and 16.1%. The lowest are the role-scoped One-Class SVM (0.7%) and the One-Class SVM on the median statistic (0.5%), and the role-scoped Isolation Forest is at 1.7%. As a false positive rate, the Random Forest raises a false alert on 0.93% of the 9 817 administrator test days and the framework on 0.14%, a 6.5-fold reduction.

The effect is again isolated by holding the algorithm fixed. Isolation Forest spends 49.4% of its alerts on administrators with a population reference

Table 7: How far down each detector's queue an analyst must read to reach each insider (user-days ranked above the insider's best malicious day, median over ten seeds, with range where it varies), and the median number of days from the insider's first malicious day to the first flagged one at the 0.1% budget. An alerts-ahead value of 0 means the insider's best day is at the top of the queue, and "never" means not flagged in any seed.

	Alerts ahead		Detection delay (days)	
Detector	**CMP2946**	**CDE1846**	**CMP2946**	**CDE1846**
Framework (personal, OCSVM)	186 (168 to 191)	85 (6 to 140)	0	29
Framework, role scope	5 626 (5 108 to 6 479)	0	never	29
Framework, iForest scorer	1 239 (726 to 2 398)	80 (40 to 218)	never	29
Population OCSVM (control)	2 175 (1 976 to 2 717)	0 (0 to 1)	never	29
Population iForest	3 595 (2 754 to 6 083)	1 414 (685 to 1 951)	never	never
Random Forest (baseline)	45 (19 to 78)	24 367 (9 093 to 77 971)	0	never
Volume floor	30 040	0	never	49

and 7.7% with a personal one. One-Class SVM goes from 20.8% to 3.5%. Expressing activity relative to the individual, not the algorithm, removes the bias.

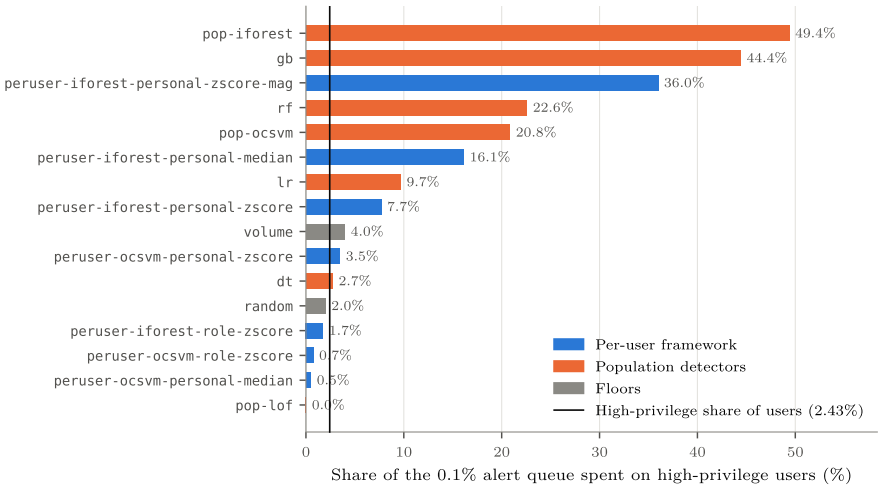


Figure 5: Share of each detector's 403 alerts spent on high-privilege users. The vertical line is their share of the test population.

The problem is not confined to the role the proposal anticipated. Executives are also unusual for good reason. The population One-Class SVM

raises a false alert on 18.7% of the test days of the user in the President role and 4.0% of those in the Vice-President role, and Isolation Forest on 15.9% and 2.9%. The framework's rates are 0.9% and 0%. A detector that only special-cases administrator roles would miss this. One that measures each user against themselves needs no list.

Two results limit the claim. First, the role-scoped framework has one of the lowest administrator shares (0.7%) but finds only 3 malicious days, and the median-statistic variant, lower still at 0.5%, finds only one, so the personal scope with z-scores is the better trade-off and not the lowest false-positive option. Second, the size channel, intended to stop very small but personally unusual days from crowding the queue, raises Isolation Forest's administrator share from 7.7% to 36.0%. It brings back the population bias the framework exists to remove, and the framework does not use it.

6.4 Comparing the Parts of the Framework

The proposal requires the anomaly algorithm to be chosen by experiment. Table 8 summarises the 45-detector comparison, which tries every combination of scope, statistic, algorithm and the size channel (36 framework variants) alongside the 9 other detectors, on the 400-user sample of Section 3.6 (40 360 test user-days, 40 alerts at 0.1%). The table shows the z-score cells and two variant rows, since the rest follow the same pattern.

Table 8: Comparison of the framework's parts on the 400-user sample. Average precision / recall at 0.1% (40 alerts), z-score statistic. A recall of 0.08 is 3 of the 38 malicious days. The population-scope cells for One-Class SVM and Isolation Forest match `pop-ocsvm` and `pop-iforest` exactly, and the LOF cell agrees with `pop-lof` to three decimals (0.00186 against 0.00187), confirming that the three scopes share one piece of code.

Anomaly Scorer	**Population**	**Role**	**Personal**
One-Class SVM	0.046 / 0.08	0.086 / 0.08	**0.292 / 0.50**
Isolation Forest	0.009 / 0.00	0.022 / 0.08	0.059 / 0.08
LOF	0.002 / 0.00	0.002 / 0.00	0.027 / 0.00
Variants of the personal scope			
Median statistic (OCSVM / iForest)	0.030 / 0.03	and	0.044 / 0.08
+ size channel (OCSVM / iForest)	0.173 / 0.37	and	0.047 / 0.08

Three conclusions follow. The reference frame matters for every algorithm. The personal scope gives the highest average precision in all three rows, six to fifteen times that of the population scope. The algorithm matters more than expected. One-Class SVM on personal z-scores is best by five times, LOF finds nothing in any setting while being the slowest model, and Isolation Forest sits in between. The median statistic, which was expected to make slow drift more visible by resisting spikes, did not help the best setting. A likely reason is that counts which are zero on most days have an IQR of zero, and raising that scale to one throws away the user's real variation.

Rolling features. Adding 7-day and 30-day rolling means of every feature, computed from past days only, gives 39 features in all and helps one scorer while hurting another. This part was rerun on the full population (Table 9). The Isolation Forest, which looks at one feature per split and copes with repeated information, gains nine times in average precision with the median statistic (the variant in Table 9) and 3.5-fold with z-scores (0.0030 to 0.0106). One-Class SVM loses most of its recall, which fits how an RBF kernel behaves when thirteen features become 39 that move together and distances lose their meaning (Section 4). The population detectors gain nothing, which is itself telling. Drift in behaviour carries information only relative to a person's own baseline. The algorithm and the feature set therefore have to be chosen together. The best setting overall is One-Class SVM on personal z-scores with the base features.

Table 9: Effect of the rolling-window features on the full population.

	Average precision		**Recall at 0.1%**	
Detector	**base**	**+ rolling**	**base**	**+ rolling**
Framework, iForest, median statistic	0.0041	**0.0378**	0.079	0.079
`peruser-iforest-personal-zscore`	0.0030	0.0106	0.079	0.079
`peruser-ocsvm-personal-zscore`	**0.0383**	0.0125	**0.553**	0.053
`volume`	0.0284	0.0281	0.053	0.053
`pop-iforest`	0.0009	0.0006	0.000	0.000

6.5 Consistency Across Random Seeds

Every comparison so far uses one seed. Table 10 and Figure 6 repeat the seven key detectors under ten seeds, with every random choice re-drawn each time.

Table 10: Mean ± standard deviation over ten seeds, full population, 0.1% budget. "Days" is the range of malicious days flagged across the seeds. "Fit" is the mean fit time in seconds.

Detector	AP	Recall	Days	Insiders	HP share	Fit (s)
`peruser-ocsvm-personal-zscore`	0.0363 ± 0.0011	**0.547 ± 0.011**	**20 to 21**	**2/2**	3.4 ± 0.2%	7.9
`peruser-ocsvm-role-zscore`	**0.0473 ± 0.0049**	0.079 ± 0.000	3	1/2	**0.5 ± 0.1%**	6.6
`peruser-iforest-personal-zscore`	0.0029 ± 0.0008	0.076 ± 0.008	2 to 3	1/2	7.4 ± 1.5%	7.4
`pop-ocsvm`	0.0360 ± 0.0063	0.079 ± 0.000	3	1/2	20.4 ± 1.7%	2.6
`pop-iforest`	0.0009 ± 0.0001	0.000 ± 0.000	0	0/2	47.5 ± 7.3%	3.3
`rf`	0.0049 ± 0.0027	0.163 ± 0.046	3 to 8	1/2	21.3 ± 2.5%	306.1
`volume`	0.0284 ± 0.0000	0.053 ± 0.000	2	1/2	4.0 ± 0.0%	0.0

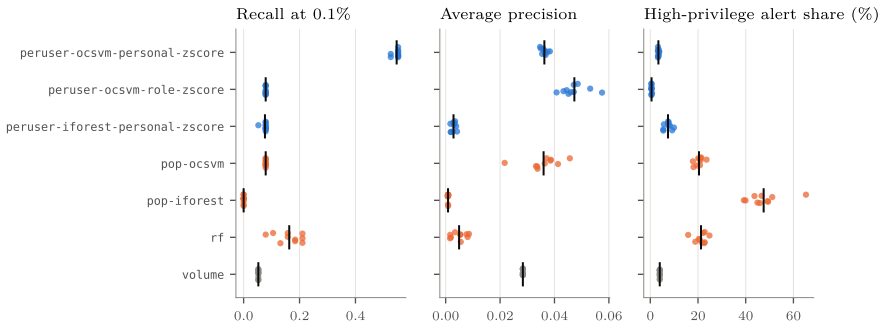


Figure 6: Each detector under ten seeds. Dots are seeds. The black tick is the mean.

The framework's result is stable. It found 20 or 21 of the 38 malicious days in every seed, and both insiders in every seed. The Random Forest ranged from 3 to 8 days, and its single-seed main (8 days) was its best draw. The paired tests confirm the picture. Seventeen of the eighteen framework-versus-detector comparisons went the same way in all ten seeds, fifteen in the framework's favour and two against it (both against the role variant), and one was mixed (average precision against `pop-ocsvm`). Each gives an exact two-sided Wilcoxon $p = 0.002$, and $p_{\text{Holm}} = 0.035$ after correcting for all eighteen, the smallest adjusted p-value a ten-seed design with eighteen tests can give. Significant below means a Holm-corrected p under 0.05, and 0.035 is 0.002 times 18. The ten-seed means in Table 10 differ slightly from the

seed-42 values of Table 6 for this reason. The framework has significantly higher recall than every other detector. It has a significantly lower high-privilege alert share than every population detector, the volume floor (whose share never varies, so a consistent 0.6 point gap passes the test) and its own Isolation Forest variant. It also has significantly higher average precision than the Random Forest, both Isolation Forest detectors and the volume floor.

Three comparisons do not favour the framework, and they limit what is claimed. Against its same-algorithm control, average precision is a tie, five seeds each way, $p = 1$. Both give one of CDE1846's attachment-burst days the maximum score (for the framework, 140 rows share the maximum score, as Section 6.10 explains, so its position in Table 7 is set by the tie-break), and that one day decides most of the average precision, so the per-user frame's advantage lies in recall and in where the second insider lands, not in the top-ranked alert. Against the role-scoped variant, the framework loses in all ten seeds on both average precision and administrator alert share, while finding seven times as many malicious days. The role reference is the stronger way to cut false alerts. The personal reference is the stronger detector.

6.6 SQ3 How Much History Is Needed

Figure 7 and Table 11 show the history-length study. The two views of stability agree. With three days of history, a third of all benign test days show some feature beyond $|z| = 3$, because a few days cannot estimate a user's spread. That share falls to 12.9% at 14 days and 8.7% at 30 days, and then flattens, reaching 5.9% with the full history. The per-user means converge in the same way. By 30 days, 99.5% of user-feature means lie within half a standard deviation of their full-history value.

Detection needs more history than stability does. Below 30 days the personal baseline finds almost nothing and is worse than simply using the role baseline (recall 0.079). At 60 active days, about three months of working days, recall jumps to 0.34 and both insiders are found. Gains continue to roughly 120 to 180 days and then level off. Below 13 days the anomaly model also has fewer than its usual 50 000 training rows (4 000 users times 13 days is just above 50 000), so the shortest histories mix the effect of profile quality with that of training-set size. From 14 days upward the training size is constant, and the trend is due to the profiles alone.

The answer to SQ3 is therefore about 30 active days for a baseline that is *stable*, and about 60 for one that is *useful*. Too little history is not merely useless, it is harmful for the wrong scorer. With three days of history the

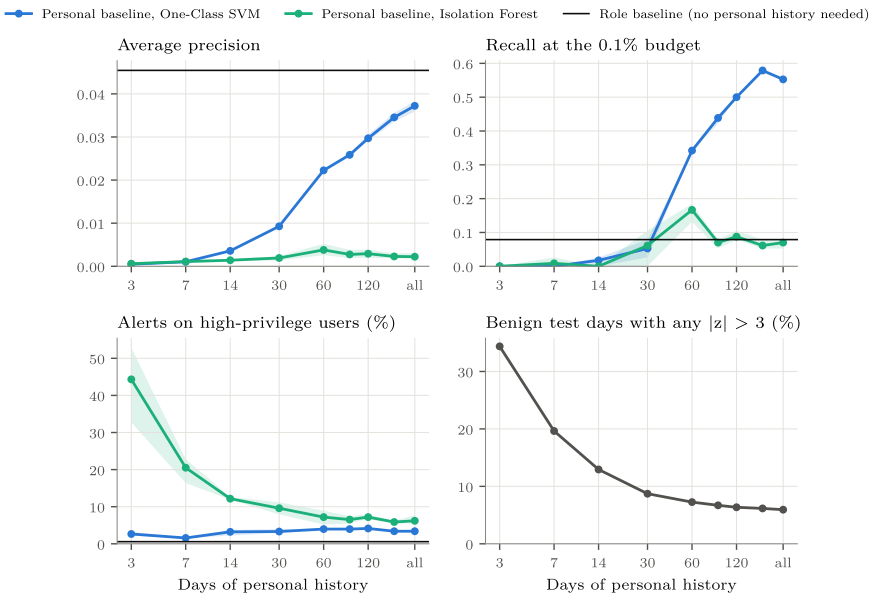


Figure 7: Detection and stability against days of personal history (most recent active days before monitoring starts). Lines are means over three seeds. Bands span the seeds.

Table 11: History-length study, One-Class SVM personal baseline (means over three seeds, so insider recall can take values other than 0, 0.5 and 1). "False-deviation days" is the share of benign test days with any feature beyond $|z| = 3$.

History	AP	Recall	Insider recall	HP share	False-deviation days
3 days	0.0005	0.000	0.00	2.7%	34.4%
7 days	0.0010	0.000	0.00	1.6%	19.6%
14 days	0.0036	0.018	0.33	3.2%	12.9%
30 days	0.0093	0.053	0.50	3.3%	8.7%
60 days	0.0223	0.342	1.00	4.0%	7.3%
90 days	0.0259	0.439	1.00	4.0%	6.7%
120 days	0.0297	0.500	1.00	4.1%	6.4%
180 days	0.0345	0.579	1.00	3.4%	6.2%
All (248)	0.0372	0.553	1.00	3.4%	5.9%
Role baseline	0.0454	0.079	0.50	0.6%	

Isolation Forest variant spends 44% of its alerts on administrators (Figure 7, bottom left), because an under-estimated spread makes naturally variable users look unusual. A short personal baseline repeats the population failure. The One-Class SVM variant is unaffected (1.6 to 4.1% throughout).

Why 14 days is too short. At 14 days, 6.8% of the user-feature pairs that vary over the full history have a standard deviation of exactly zero, because the user showed no variation in that feature within two working weeks, and the scale is then floored at one. At 60 days this falls to 2.3%. At 14 days, one in ten per-user means still sits more than 0.4 standard deviations from where the full history places it. A 14-day profile therefore describes two weeks of behaviour, not the person's normal, and 12.9% of ordinary days are scored as deviations against it, more than twice the 5.9% of the full history. The practical reading is that an employee should not be judged against their own first two weeks, in which a routine is still forming. This reading is an interpretation. The study keeps each user's most recent N active days, so it imitates a short logging history rather than a new employee. What the measurements show is that the 14-day cold-start default is too low, and that until about 60 days are available the role baseline, which needs no personal history, is both safer (0.6% administrator share) and more effective (recall 0.079 against 0.018). Raising the threshold has a cost the main experiment hides. Every r6.2 user has at least 21 training days, so the 14-day threshold never fell back to the role group, but under a 60-day threshold 43 of the 4 000 users (1.1%) would have been scored against their role instead.

6.7 SQ4 Computing Cost

Table 12 reports measured cost on the laptop. The per-detector rows are means over the ten seeds of Section 6.5, because single-run elapsed times on a laptop proved unreliable. The main comparison run on 21 October 2026 and again on 29 October gave the same detection measures, but fit times that differed by up to two times (7.4 against 15.8 seconds for the framework, 289 against 544 seconds for the Random Forest), depending on whatever else the laptop was doing. The ratio between the two detectors, 39 in the seed study and 34 to 39 in the two single runs, is the reliable quantity.

Building 4 000 personal profiles, 46 role profiles and the population profile from 990 765 rows, and fitting the anomaly model, takes about eight seconds. Only one anomaly model is fitted regardless of the number of users, so cost grows with rows and not with users. The framework's 7.9 seconds against 2.6 for the same One-Class SVM with a population reference is the cost

Table 12: Computational cost on the full population (990 765 training and 403 245 test user-days). Fit times are mean ± standard deviation over ten seeds. "ms/row" is the mean scoring time per test user-day. "Peak" is the most memory used by fit and score together. The one-off rows are as recorded in the pipeline's documentation when they were run, not from a run record.

Detector	**Fit (s)**	**ms/row**	**Peak (GB)**
Framework (personal, One-Class SVM)	7.9 ± 0.5	0.027	1.3
Framework, role scope	6.6 ± 0.5	0.025	0.9
Framework, Isolation Forest scorer	7.4 ± 0.3	0.007	1.3
Population One-Class SVM	2.6 ± 0.1	0.023	0.2
Population Isolation Forest	3.3 ± 0.3	0.005	0.2
Random Forest (proposal baseline)	306.1 ± 21.1	0.0015	1.9
Volume floor	0.0	0.0000	0.04
One-off: raw logs to user-day table (93 GB), 12 workers	198		not recorded
One-off: the same with 4 workers	900		2.8
One-off: user-day table to frozen dataset	3		

of building the profiles and the deviation vectors. The Random Forest is 39 times slower to train, because SMOTE adds 990 753 made-up malicious rows to the six real ones, doubling the training set to 1 981 518 rows. The framework is slower than the Random Forest to *score*, at 0.027 ms per user-day against 0.0015 ms, but a 4 000-person organisation produces about 4 000 user-days a day, which score in about a tenth of a second. The expensive step is the one-off reduction of the raw logs, which reads 93 GB in 3.3 minutes with 12 workers, or 15 minutes with at most 2.8 GB of memory when limited to 4. Per-user baselining is therefore feasible on consumer hardware, and cheaper to train than the population baseline it replaces.

6.8 SQ5 Context for the Analyst

A user study of investigation time was outside the project's reach. SQ5 is therefore answered with two measurable stand-ins. The first is alerts ahead, how far down the ranked queue an analyst must read before reaching each insider (Section 6.2). The second, reported here, is whether the reason each system gives for a malicious day names what the insider actually did.

Figure 8 scores the top three features of each explanation against the answer key's ground truth for all 38 malicious test days. SHAP is scored in two readings, ranked by the size of the contribution ignoring its sign, and restricted to features that push towards an alert. The framework's explanation names a feature the malicious events actually changed as its top reason on 94.7% of days, against a random-explanation rate of 29.6% (the chance formula of Section 3.5 with one named feature). With the same method but a population reference the rate is 52.6%, and against the role group 39.5%. The explanation method is the same in all three. Only the reference frame differs. On the 21 malicious days the framework actually flagged, its top-named reason is correct every time. Precision@3, the share of the three named features that are relevant, is 0.64 for the framework, 0.53 with a population reference, 0.44 with a role reference and 0.26 for the positive SHAP reading, against a chance rate of 0.30.

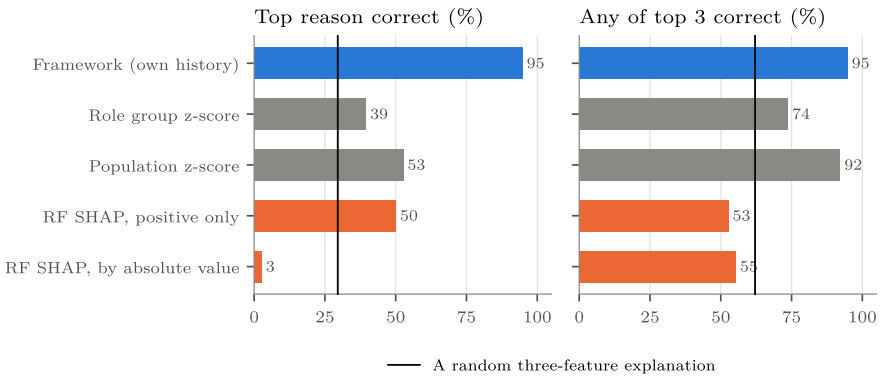


Figure 8: How often each explanation names a feature the insider's malicious events actually changed that day, over the 38 malicious test days. The horizontal line is the expected rate of a random three-feature explanation.

How the Random Forest's SHAP explanations perform depends on how they are read. Ranked by size ignoring sign, they name a relevant feature first on only 2.6% of malicious days, far below the 29.6% of a random explanation, and on none of the 8 days the forest itself flagged. The forest assigns

most low-and-slow days a probability near zero, so its largest SHAP values are reasons *against* an alert. Reading only the features that push towards an alert raises the rate to 50.0% over all malicious days and to 87.5% on the forest's own 8 alerts. When the forest does raise an alert, its explanation names a relevant feature first on 7 of its 8 alerts, provided it is read for positive contributions. The dashboard therefore shows only the positive contributions.

Two differences from the framework remain. The first is coverage. The framework flagged 21 malicious days and its top-named reason was right for all 21, while the forest raised 8, and across all 38 malicious days even the positive reading of SHAP includes a relevant feature in its top three less often than a random explanation would (52.6% against 62.1%), because the forest learned the malicious class from the burst insiders. The second is context. A SHAP value is a contribution to a probability and says nothing about what is normal for the person. A deviation says what the value is today, what it usually is for this user, and how far apart the two are.

The insider-level breakdown mirrors the detection result. On CDE1846's days, which are extreme for anyone, every z-score explanation is correct. On CMP2946's, the framework's top reason is correct 93% of the time, against 38% with a population reference and 21% with a role reference. Personal context matters for exactly the behaviour that is unusual only for the person.

Table 13 shows what this means at the analyst's desk. On CMP2946's highest-scored malicious day, the framework reports that web requests rose to 173, from a user who made exactly 162 on every training day. With a spread of zero, floored at one, that is a deviation of 11. To the population this is a 1.8-standard-deviation day, ordinary among 4 000 users. On CDE1846's, it reports eight file operations against a usual zero (12 standard deviations), which the population reference rates at 1.4. That is the trace of the insider opening files on other users' machines, alongside the emailed attachments that every reference frame sees. The forest flagged neither day, and its SHAP values for both are small and negative.

The answer to SQ5 is that a per-user deviation tells the analyst what changed, by how much relative to this person's own normal, and what their normal is. Measured against ground truth, that reason was right on 95% of malicious days and on every alert the framework raised. The Random Forest's SHAP explanations were right for most of its own, far fewer, alerts when read for positive contributions, but no better than chance across the malicious days as a whole, and they carry no notion of the user's normal. Whether this shortens investigations in practice needs the user study proposed as future work.

Table 13: What each system tells the analyst about the two insiders' highest-scored malicious days. "Usual" and "p90" are the user's own median and 90th-percentile day. Rows are the features the framework names first, plus, for CDE1846, the file feature discussed in the text. A negative SHAP value is a reason against an alert.

Feature	Today	Usual	p90	z (own)	z (population)	RF SHAP
CMP2946, 18 February 2011 (job hunting, before the thumb-drive phase)						
`n_http_events`	173	162	162	**+11.0**	+1.8	−0.04
`n_email_sent`	12	6	8	+3.2	+4.4	−0.05
`n_distinct_domains`	27	23	26	+1.2	+0.3	−0.00
CDE1846, 25 April 2011 (another user's PC, files emailed to a personal address)						
`total_email_size`	48.6 MB	1.4 MB	5.1 MB	**+19.4**	+21.5	−0.03
`n_email_attachments`	48	1	6	+16.7	+20.0	−0.03
`n_email_external`	18	1	3	+15.2	+11.7	−0.07
`n_email_sent`	23	3	5	+13.8	+9.5	−0.04
`n_file_events`	8	0	1	**+12.2**	+1.4	−0.07

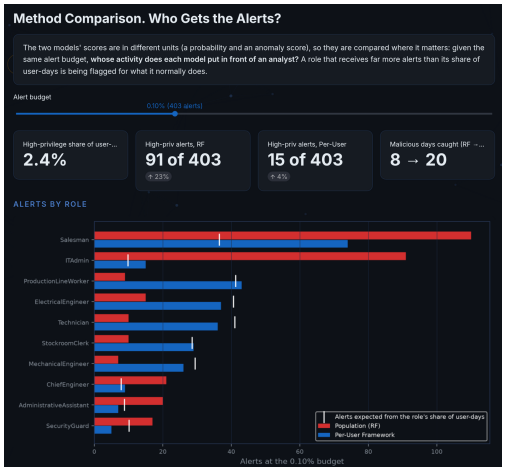
6.9 The Dashboard

The proposal promised a prototype, a first working version, that a non-technical security manager could use, and this is the product that a reader can try. The Streamlit dashboard has six pages. *Risk Explorer* plots a user's daily risk, the 0 to 1 display score of Section 3.2, under both the framework and the Random Forest, with each model's alert line (the score at which its 0.1% budget is cut), and explains every flagged day against the user's own typical and 90th-percentile day (Figure 9a). For the Random Forest it shows the SHAP contributions that push towards an alert. *Method Comparison* shows who each model spends its alert budget on, role by role, against each role's share of user-days (Figure 9b). *Evaluation* scores the whole test window at a chosen budget, *Findings* presents the saved results reported in Section 6, and *Overview* and *About* complete the set. The dashboard's two models are trained on the full population with the defaults of Table 3, except that its One-Class SVM uses a contamination of 0.03 rather than 0.02, a leftover of the training script's own default that changes the counts only slightly. At the 0.1% budget it shows about three alerts a day. Its counts therefore differ slightly from Section 6, 20 malicious days caught instead of 21 and 15 administrator alerts instead of 14. The dashboard runs as a packaged copy of

the software with the trained models attached read-only, and is served over HTTPS from Amazon Web Services (AWS) at the address on the title page.



(a) Risk Explorer for CMP2946. The framework (blue) stays at or above its alert line on the malicious days, and each alert names what changed against the user's own normal.



(b) Method Comparison. Of 403 alerts, the Random Forest spends 91 on IT administrators, who are 2.4% of user-days, and the framework 15.

Figure 9: The deployed dashboard (full population, 0.1% budget).

6.10 Unexpected and Negative Results

A no-model detector beats most baselines. Ranking user-days by total activity, with no model at all, matches Logistic Regression's average precision (0.0284) and beats the Random Forest's (0.0077). CMP2946's (scenario 2) thumb-drive days are simply busy days. Results on this dataset that do not report such a no-model detector should be read with caution.

Two supervised baselines fail outright. Grown until every training row is classified, the Decision Tree gives exactly two distinct scores on the test window, probability 1 for two rows and 0 for the other 403 243. It therefore has no ranking, and 99.5% of its budget is a coin toss. Gradient Boosting reaches a ROC-AUC of 0.379, worse than random. Both follow from learning a malicious class from six burst examples and being tested on low-and-slow behaviour.

Two measures mislead in opposite ways. `pop-iforest` has a ROC-AUC of 0.935, the second highest in Table 6, yet finds no malicious day and spends half its queue on administrators. It orders the benign mass well and the top badly. On the other hand, the role-scoped framework has the highest average precision (0.0433) but finds 3 days to the personal framework's 21, because average precision is decided mostly by the very top of the ranking. No single number suffices, which is why recall and insider-level recall at the budget, the budget sweep and queue depth are all reported.

Many days share the top score. Far from every training point the model kept, the RBF kernel sum is zero and the One-Class SVM's score equals its constant ρ, its highest possible value. In the test window, 140 user-days receive exactly this maximum score, and a further seven lie within 10^{-9} of it. Three of these 147 days are malicious, one of them in the exactly tied group. At the 0.1% budget all 147 are included and the effect is invisible, but at 40 alerts the selection is a seeded coin toss among the 140 tied rows (tie fraction 1.0, every one of the 40 alerts chosen by the tie-break) that happened to draw no malicious day. Ordering this top group by the sum of the absolute deviations in the vector would fix this, and is left as future work.

6.11 Limits of the Evidence

Only two insiders. The test window contains 38 malicious days, but they belong to only two people. The seed study controls for chance inside the algorithms. It cannot control for which insiders the dataset happens to contain. Insider-level claims rest on $n = 2$, and day-level recall moves in steps of 2.6 points. The results are evidence about these two low-and-slow scenarios, not an estimate of performance on insiders in general.

How the algorithm was chosen. The date split puts every burst insider in the training window, so the supervised baselines learn from six examples of a different kind of attack. This puts them at a disadvantage, and is why the population *unsupervised* controls, which see no labels either, are the fairer comparison. The framework beats those too. More seriously, the anomaly algorithm was chosen on the 400-user sample, whose test window contains the same two insiders as the full test window. The choice of One-Class SVM over the Isolation Forest the framework was first built with is therefore informed by test-window behaviour, and the main recall is probably too high. With the Isolation Forest scorer the framework was first built with, the personal reference frame still cuts the administrator alert share from 49.4% to 7.7%

and finds 3 malicious days and one insider where the population control finds none, but it comes nowhere near the One-Class SVM's 21 days.

What the measures leave out. Labels are day-level, so a flagged day counts as a hit even if the analyst would need to find the malicious events within it. The alert budget is applied over the whole test window after the fact. A system in use must choose each day's cut-off without seeing the future. The high-privilege subgroup is defined by role and in r6.2 is exactly one role. The only high-privilege insider (PLJ1771, scenario 3) acts in the training window, so this evaluation cannot show whether per-user scoring would also suppress a *malicious* administrator.

Generated data. CERT r6.2 is generated by a program from models of one made-up organisation [11]. Its users are very regular. CMP2946 made exactly 162 web requests on every training day. Such sameness flatters personal baselines, because small departures stand out. Real logs are expected to be noisier, which this study did not test, so results on real logs, and on other CERT releases, may be weaker.

6.12 The Proposal Reviewed

The research proposal set out a problem statement, five sub-questions, a three-part framework, a dataset, a baseline, a set of measures, a plan for the code, a schedule and a risk register. The paragraphs below record where what was built departs from the plan and why. Tables 14 and 15 collect every design choice that did not work.

Problem statement and literature. The problem statement and the five sub-questions were carried into this report with only light rewording. The proposal's hypothesis ended "without significant loss in overall accuracy". Accuracy is not a usable measure on this dataset, since a detector that flags nothing scores 99.99% on a test window in which 38 of 403 245 rows are malicious, so the report's hypothesis instead requires a cost that is feasible on consumer hardware. Both failures the proposal predicted appeared in the data (Sections 6.2 and 6.3). A third it did not foresee, the over-flagging of executives, appeared as well and was removed by the same reference frame.

Checking each of the proposal's sources while writing this report found six errors, listed here so that the reader does not rely on the proposal's versions of them. The paper cited as Noever's "Machine learning classifiers and the insider threat" (arXiv:1908.05895) is "Classifier Suites for Insider

Table 14: Defects that produced numbers of the right shape and the wrong meaning, each now locked in by a test.

Choice	Why it failed	Replacement	Section
Loading the raw logs into memory	93 GB of logs against 32 GB of memory	Read the logs once, in pieces	3.1
Reading every release's answer key	239 users labelled as insiders in a release with five	Filter to r6.2	3.1
Blaming every user named in the answer key	The 24 recipients of the scenario 3 email were labelled attackers	Blame only the named insider	3.1
Window-filled labels	95 user-days labelled where 44 are malicious	Exact day labels	3.1
Four feature definitions	Constant or broken on real data	Corrected and locked in by tests	3.1
Rescaled and capped risk score	Every row above the training range tied at 1.0, the whole top of the ranking	Model's own score for ranking, 0 to 1 score for display	3.2
Flagging every row tied at the cut	Random Forest flagged all 40 360 cohort test rows against a 1% budget of 404	Seeded tie-break and tie fraction	3.4
First tie-fraction formula	Reported a non-zero tie share even when no rows were tied	Off-by-one fixed	3.4
SMOTE seeded with 42 always	Seed study would not have re-drawn it	Takes the run's seed	3.7

Threat Detection" (arXiv:1901.10948) [4]. The paper cited as Yuan et al., IEEE Transactions on Information Forensics and Security 2021, with a 31 per cent false positive reduction from role normalisation, is a 2018 conference paper on an LSTM and CNN classifier [27] that makes no such claim. The paper cited as Le and Zincir-Heywood 2020, with a user study in which SHAP raised analyst agreement by 23 per cent, is a 2018 workflow paper [5] with no SHAP user study. The paper cited as Liu, Xu, Shu and Liu (2019) in Computers and Security, with a 14 percentage point recall gain for per-user baselines, could not be found. The 197-day detection time the proposal attributed to the IBM 2024 report is not what that report gives, which is 287 days [1]. The forensic readiness paper the proposal cited as a 2025

Table 15: Modelling choices the controlled comparison rejected.

Choice	Why it failed	Replacement	Section
Median statistic	AP 0.030 against 0.292 for the z-score on the sample, zero IQRs raised to one	z-score	6.4
Size channel	Administrator alert share 7.7% to 36.0%	Not used	6.3
Rolling-window features with the One-Class SVM	Recall 0.553 to 0.053	Base features	6.4
LOF as scorer	No malicious day found in any setting, slowest model	One-Class SVM	6.4
Isolation Forest as default scorer	3 malicious days against 21	One-Class SVM	6.4
14-day cold-start threshold	Recall 0.018 at 14 days against 0.342 at 60	About 60 days recommended	6.6
SHAP ranked by size ignoring sign	Top reason correct on 2.6% of malicious days, below chance	Positive contributions only	6.8

journal article by Shoderu, Zuva and Fadahunsi is a 2026 conference paper by Shoderu, Baror, Modupe and Makura [28]. The report relies on none of the numbers from those entries. The proposal's other sources were checked and kept, its two industry sources for alert fatigue and UEBA were replaced by a peer-reviewed interview study [6] and the vendor's own documentation [7], and the autoencoder (a neural network that learns to reproduce its input) it reviewed [29] was not part of the planned comparison and was not built.

Dataset, framework and measures. The proposal described r6.2 as 1 000 users over 18 months with logon, file, device, print and email logs and psychometric (personality test) scores. The release has 4 000 users over 516 days, no print log, and a web log the proposal did not mention, which is the largest file and the source of three of the thirteen features (Table 1). Psychometric scores were excluded by the scope. The proposal's "computational infeasibility" risk appeared at data loading rather than at profile building, and was handled by reading the logs once (Section 3.1). The proposal's "configurable historical window" became the full training history, and its length was studied as the variable of SQ3, which is what the risk register said to do if history proved too short. The Anomaly Scorer scores a user-day

rather than "a new log record", because every feature is a daily count and a per-user-day table is what makes a 93 GB log fit a laptop. The median statistic, the size channel and the rolling-window features the risk register reserved for a weak result were all evaluated and rejected (Table 15). Beyond the proposal's Random Forest, three more supervised classifiers, three population anomaly controls and two no-model detectors were added so that every difference could be traced to its cause. The proposal's measures are all reported, at an alert budget rather than a fixed cut-off because the detectors' scores are not on a common scale (Section 3.4), and its high-privilege roles (database administrators, system operators and security analysts) do not exist in r6.2, so the subgroup is the ITAdmin role.

Schedule and risks. The proposal scheduled the work from February to October 2026 in five phases, with the framework built in May and June, the evaluation in July, and the prototype and report in August and September. The code history records a first prototype on 24 August 2026, the data pipeline and first tests on 7 and 8 September, the three follow-up studies and a first full report on 13 October, and the deployment and documentation between 21 and 24 October. The framework, the evaluation and the prototype were therefore delivered later than scheduled, with phases 2 to 4 compressed into about seven weeks. The risk register's response to delay was to drop one of the three algorithms. That was not needed, and all three were evaluated in every scope and statistic. The proposal also promised to note any experiment that could not be completed on consumer hardware. There were two, loading the raw logs into memory and fitting the One-Class SVM and LOF on all 990 765 rows, and Sections 3.1 and 3.2 give how each was resolved, the first without changing any result and the second by fitting the anomaly model on a 50 000-row sample.

What did not work. Tables 14 and 15 list every design choice that was tried and abandoned, with the measurement that ruled it out and what replaced it. The first holds the defects that produced numbers of the right shape and the wrong meaning, each now locked in by a test. The second holds the modelling choices the controlled comparison rejected.

7 Conclusion

This research asked whether a lightweight per-user behavioural baseline could detect insider activity that a population-level detector misses, while spending fewer alerts on users who are busy for good reason, on hardware a resource-constrained organisation already owns. It was evaluated on the full CERT r6.2 dataset against the proposal's Random Forest, against the same anomaly algorithms with a population reference, and against no-model detectors.

Answers to the sub-questions.

- **SQ1 (low-and-slow detection).** At a 0.1% alert budget the per-user baseline found 21 of 38 low-and-slow malicious days and both insiders, against 8 days (3 to 8 across ten seeds) and one insider for the Random Forest and 3 days for the same algorithm with a population reference. The gain is concentrated on the insider whose behaviour was unusual for themselves but ordinary for the organisation.
- **SQ2 (high-privilege false positives).** The Random Forest, the population Isolation Forest, the population One-Class SVM and gradient boosting spent 21 to 49% of their alerts on IT administrators, who are 2.4% of user-days. The framework spent 3.5%. Holding the algorithm fixed shows that the reference frame, not the model, removes the bias, for executives too.
- **SQ3 (history needed).** A personal baseline is stable after about 30 active days and useful after about 60, so the cold-start threshold should be about 60 days, not the 14-day default. Two weeks of activity is not yet a person's normal, and until 60 days are available the role baseline is the fairer and the more effective reference.
- **SQ4 (cost).** The framework trains on 990 765 user-days in about eight seconds on a laptop, 39 times faster than the SMOTE-balanced Random Forest, and the one-off reduction of 93 GB of logs takes 3.3 minutes in limited memory.
- **SQ5 (analyst context).** A per-user deviation tells the analyst what changed, by how much relative to this person's normal, and what that normal is. Its top-named reason was correct on 94.7% of malicious test days, against 52.6% with a population reference and at most 50.0% for the Random Forest's SHAP values, and both insiders sit within its first 200 alerts. Whether this shortens real investigations needs a user study.

Contributions. The work contributes a working framework and data pipeline that others can rerun, a controlled evaluation design that other work on this dataset can reuse, three follow-up studies, and a record of the negative results and of every change from the proposal.

Limitations and future work. The evidence comes from one synthetic dataset and two test-window insiders, the anomaly algorithm was selected on data containing those insiders, the only malicious administrator falls outside the test window, and the alert budget is set after the fact (Section 6.11). Five directions follow directly from the findings. A daily alert budget that chooses each day's alerts from that day's scores, as a system in use must. Ordering the One-Class SVM's top-scored group by the size of the deviation vector. Baselines that switch from role to personal reference at the roughly 60 days SQ3 identifies. Selecting the algorithm on one CERT release and evaluating on others (r4.2, r5.2), or on real logs collected with permission. And a user study of whether the per-user explanations reduce investigation time.

Conclusion. Judging each employee against their own normal, rather than the organisation's, found a slow insider (CMP2946) that the same algorithm with a population reference ranked thousands of alerts deep, kept busy administrators and executives out of the alert queue, and ran in seconds on a laptop. It is not a replacement for a security operations centre, but evidence that the core idea of commercial behaviour analytics is within reach of organisations that cannot buy one.

References

[1] IBM Security, "Cost of a data breach report 2024," tech. rep., IBM Corporation, 2024. [Online]. Available: https://www.ibm.com/reports/data-breach.

[2] D. M. Cappelli, A. P. Moore, and R. F. Trzeciak, *The CERT Guide to Insider Threats: How to Prevent, Detect, and Respond to Information Technology Crimes*. Addison-Wesley Professional, 2012. ISBN 978-0-321-81257-5.

[3] I. Homoliak, F. Toffalini, J. Guarnizo, Y. Elovici, and M. Ochoa, "Insight into insiders and IT: A survey of insider threat taxonomies, analysis, modeling, and countermeasures," *ACM Computing Surveys*, vol. 52, no. 2, pp. 30:1–30:40, 2019. doi:10.1145/3303771.

[4] D. Noever, "Classifier suites for insider threat detection." arXiv preprint arXiv:1901.10948, 2019.

[5] D. C. Le and A. N. Zincir-Heywood, "Evaluating insider threat detection workflow using supervised and unsupervised learning," in *IEEE Security and Privacy Workshops (SPW)*, pp. 270–275, 2018. [Online]. Available: https://ieeexplore.ieee.org/document/8424659.

[6] B. A. Alahmadi, L. Axon, and I. Martinovic, "99% false positives: A qualitative study of SOC analysts' perspectives on security alarms," in *31st USENIX Security Symposium*, pp. 2783–2800, 2022. [Online]. Available: https://www.usenix.org/conference/usenixsecurity22/presentation/alahmadi.

[7] Microsoft, "Advanced threat detection with user and entity behavior analytics (UEBA) in Microsoft Sentinel." Microsoft Learn, 2026. [Online]. Available: https://learn.microsoft.com/en-us/azure/sentinel/identify-threats-with-entity-behavior-analytics (accessed October 2026).

[8] A. Tuor, S. Kaplan, B. Hutchinson, N. Nichols, and S. Robinson, "Deep learning for unsupervised insider threat detection in structured cybersecurity data streams," in *AAAI Workshop on Artificial Intelligence for Cyber Security*, pp. 224–231, 2017. arXiv:1710.00811.

[9] D. C. Le, N. Zincir-Heywood, and M. I. Heywood, "Analyzing data granularity levels for insider threat detection using machine learning,"

IEEE Transactions on Network and Service Management, vol. 17, no. 1, pp. 30–44, 2020. doi:10.1109/TNSM.2020.2967721.

[10] B. Lindauer, "Insider threat test dataset." Carnegie Mellon University, KiltHub. doi:10.1184/R1/12841247.v1, 2020.

[11] J. Glasser and B. Lindauer, "Bridging the gap: A pragmatic approach to generating insider threat data," in *IEEE Security and Privacy Workshops (SPW)*, pp. 98–104, 2013. doi:10.1109/SPW.2013.37.

[12] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and É. Duchesnay, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011. [Online]. Available: `https://jmlr.org/papers/v12/pedregosa11a.html`.

[13] S. Yuan and X. Wu, "Deep learning for insider threat detection: Review, challenges and opportunities," *Computers & Security*, vol. 104, p. 102221, 2021. doi:10.1016/j.cose.2021.102221.

[14] P. J. Rousseeuw and M. Hubert, "Robust statistics for outlier detection," *WIREs Data Mining and Knowledge Discovery*, vol. 1, no. 1, pp. 73–79, 2011. doi:10.1002/widm.2.

[15] V. Chandola, A. Banerjee, and V. Kumar, "Anomaly detection: A survey," *ACM Computing Surveys*, vol. 41, no. 3, pp. 15:1–15:58, 2009. doi:10.1145/1541880.1541882.

[16] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation forest," in *Proceedings of the IEEE International Conference on Data Mining (ICDM)*, pp. 413–422, 2008. doi:10.1109/ICDM.2008.17.

[17] M. M. Breunig, H.-P. Kriegel, R. T. Ng, and J. Sander, "LOF: Identifying density-based local outliers," in *Proceedings of the ACM SIGMOD International Conference on Management of Data*, pp. 93–104, 2000. doi:10.1145/342009.335388.

[18] B. Schölkopf, J. C. Platt, J. Shawe-Taylor, A. J. Smola, and R. C. Williamson, "Estimating the support of a high-dimensional distribution," *Neural Computation*, vol. 13, no. 7, pp. 1443–1471, 2001. doi:10.1162/089976601750264965.

[19] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic minority over-sampling technique," *Journal of Artificial Intelligence Research*, vol. 16, pp. 321–357, 2002. doi:10.1613/jair.953.

[20] J. Davis and M. Goadrich, "The relationship between precision-recall and ROC curves," in *Proceedings of the 23rd International Conference on Machine Learning (ICML)*, pp. 233–240, 2006. doi:10.1145/1143844.1143874.

[21] T. Saito and M. Rehmsmeier, "The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets," *PLoS ONE*, vol. 10, no. 3, p. e0118432, 2015. doi:10.1371/journal.pone.0118432.

[22] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, pp. 4765–4774, 2017. arXiv:1705.07874.

[23] S. M. Lundberg, G. Erion, H. Chen, A. DeGrave, J. M. Prutkin, B. Nair, R. Katz, J. Himmelfarb, N. Bansal, and S.-I. Lee, "From local explanations to global understanding with explainable AI for trees," *Nature Machine Intelligence*, vol. 2, no. 1, pp. 56–67, 2020. doi:10.1038/s42256-019-0138-9.

[24] F. Wilcoxon, "Individual comparisons by ranking methods," *Biometrics Bulletin*, vol. 1, no. 6, pp. 80–83, 1945. doi:10.2307/3001968.

[25] J. Demšar, "Statistical comparisons of classifiers over multiple data sets," *Journal of Machine Learning Research*, vol. 7, pp. 1–30, 2006. [Online]. Available: `https://jmlr.org/papers/v7/demsar06a.html`.

[26] S. Holm, "A simple sequentially rejective multiple test procedure," *Scandinavian Journal of Statistics*, vol. 6, no. 2, pp. 65–70, 1979. [Online]. Available: `https://www.jstor.org/stable/4615733`.

[27] F. Yuan, Y. Cao, Y. Shang, Y. Liu, J. Tan, and B. Fang, "Insider threat detection with deep neural network," in *Computational Science – ICCS 2018*, vol. 10860 of *Lecture Notes in Computer Science*, Springer, 2018. doi:10.1007/978-3-319-93698-7_4.

[28] G. Shoderu, S. Baror, A. Modupe, and S. Makura, "Digital forensic readiness to mitigate insider threats in the SaaS cloud environment," in *Proceedings of the 21st International Conference on Cy-*

ber Warfare and Security (ICCWS), vol. 21, pp. 427–438, 2026. doi:10.34190/iccws.21.1.4508.

[29] S. Hawkins, H. He, G. Williams, and R. Baxter, "Outlier detection using replicator neural networks," in *Data Warehousing and Knowledge Discovery (DaWaK)*, vol. 2454 of *Lecture Notes in Computer Science*, pp. 170–180, 2002. doi:10.1007/3-540-46145-0_17.